

Context Windows

Why a model can only read so much at once · video 2:21

This lesson answers one question: why can a model only read so much at once, and what happens when a chat outgrows it? The **context window** is the maximum number of tokens per request, prompt plus reply. It exists because of **position** (trained lengths), **attention** (work grows with the square of the length) and **memory** (the KV cache). When a chat overflows, apps **drop the oldest turns, summarise** them or **retrieve** only what matters.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 7).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. The context window	▶ 0:08-0:46	3	☐
2. Why there is a limit	▶ 0:46-1:21	4	☐
3. When the chat overflows	▶ 1:21-1:44	3	☐
4. Trimming in code, and practical lessons	▶ 1:44-2:19	3	☐

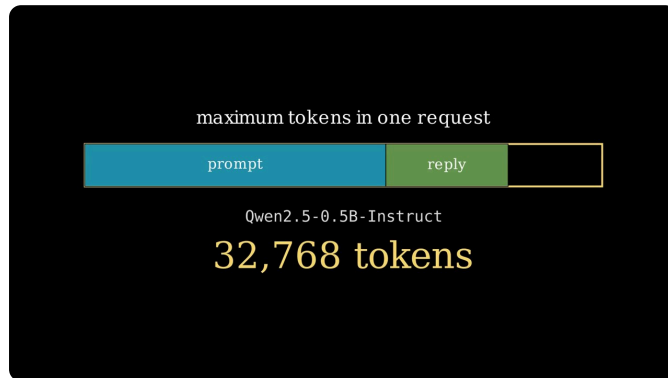
Before you start: episode 1 of this series (the whole chat is resent on every turn). How LLMs Work episodes 4 (position), 5-6 (attention) and 13 (the KV cache) explain the three reasons for the limit. Code: `code/p02_context_window` in the repository.

CONCEPT 1 OF 4

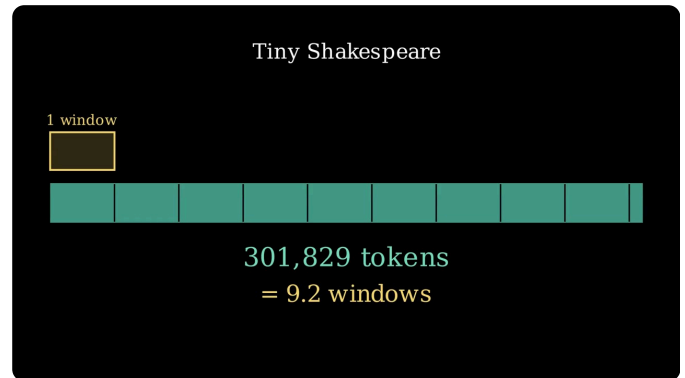
The context window

▶ Video 0:08–0:46

Every message resends the whole chat (episode 1), so the text the model reads keeps growing. But a model can only read so much at once. The **context window** is the maximum number of tokens the model can handle in **one request**: the **prompt plus the reply** it writes. For Qwen2.5-0.5B-Instruct, the model in our code, it is **32,768 tokens**.



▶ 0:34 The window holds the prompt plus the reply: 32,768 tokens for Qwen2.5-0.5B-Instruct.



▶ 0:45 Tiny Shakespeare is 301,829 tokens: 9.2 full windows.

That sounds like a lot, but Tiny Shakespeare, the collection of plays the tiny GPT of How LLMs Work learned from, is **301,829 tokens**: more than nine full windows.

KEY IDEA

The window is a hard limit per request, and the reply has to fit in it too.

Check 1: The context window

Answer all 3 questions before looking at the answers. Pass mark: all correct.

1.1 MULTIPLE CHOICE

What has to fit inside the context window?

- A. Only your newest message
- B. Only the model's reply
- C. The whole prompt (system, earlier turns, documents) plus the reply
- D. The model's training data

1.2 CALCULATION

A prompt is 30,000 tokens long. With a 32,768-token window, what is the longest reply the model can write?

1.3 CALCULATION

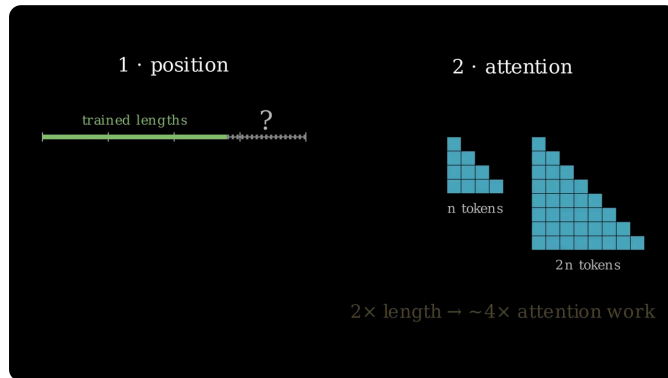
Tiny Shakespeare is 301,829 tokens. How many 32,768-token windows is that, to one decimal place?

CONCEPT 2 OF 4

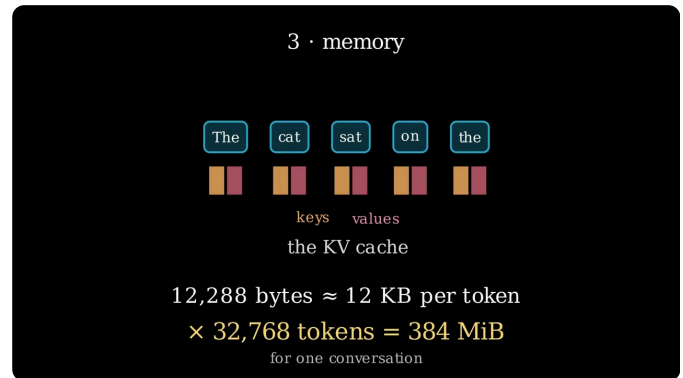
Why there is a limit

▶ Video 0:46–1:21

Three reasons. **Position**: the model was trained on sequences up to a certain length, and positions beyond that are unfamiliar. **Attention**: every new token looks back at every token before it, so **doubling the length roughly quadruples** the attention work (the triangle of token pairs grows with the square of the length).



▶ 1:03 Position: lengths beyond training are unfamiliar. Attention: double the length, about four times the work.



▶ 1:19 Memory: the KV cache keeps keys and values for every token: 12,288 bytes per token, 384 MiB for a full window.

Memory: for each token in the window the model keeps its keys and values, the **KV cache** (How LLMs Work, episode 13). In this small model that is 2 (K and V) × 24 layers × 2 KV heads × 64 numbers × 2 bytes = **12,288 bytes per token**, so a full window takes **384 MiB**, for a single conversation.

KEY IDEA

Longer contexts cost more in every way: unfamiliar positions, quadratic attention work and linear KV-cache memory.

Check 2: Why there is a limit

Answer all 4 questions before looking at the answers. Pass mark: all correct.

2.1 MULTIPLE CHOICE

A request grows from 1,000 to 2,000 tokens. Roughly how does the attention work change?

- A. It stays the same
- B. It doubles
- C. It roughly quadruples
- D. It grows tenfold

2.2 CALCULATION

The KV cache of this model takes 12,288 bytes per token. How many MiB for a 8,192-token conversation? (1 MiB = 1,048,576 bytes)

2.3 TRUE OR FALSE

“The KV cache memory grows with the square of the context length.”

True ☐ False ☐

2.4 SHORT ANSWER

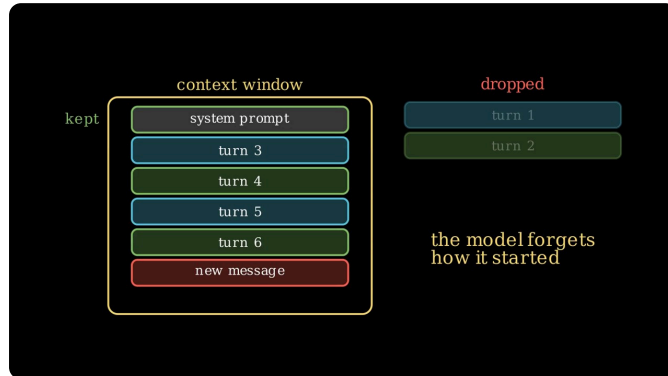
A server holds 10 full-window conversations of this model at once. How much KV-cache memory is that?

CONCEPT 3 OF 4

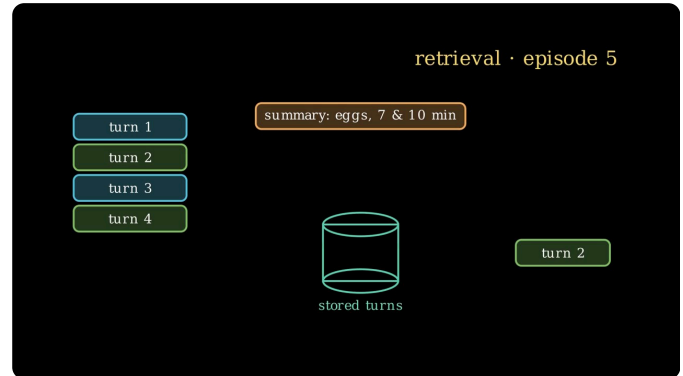
When the chat overflows

▶ Video 1:21-1:44

When a chat outgrows the window, something has to go. The usual fix: **keep the system prompt** and **drop the oldest turns**. The model simply forgets how the conversation started.



▶ 1:32 The usual fix: keep the system prompt and drop the oldest turns.



▶ 1:43 Smarter apps summarise old turns, or store them and retrieve only what matters.

Smarter apps **summarise** the old turns into a short note, or **store** them and bring back only the pieces that matter: that is **retrieval**, which we build in episode 5.

KEY IDEA

Dropped text is gone for the model. Summaries and retrieval keep the important parts inside the window.

Check 3: When the chat overflows

Answer all 3 questions before looking at the answers. Pass mark: all correct.

3.1 MULTIPLE CHOICE

Why does the trimming code never drop the system prompt?

- A. It cannot be tokenized
- B. It sets the model's role and rules for every reply
- C. It is always the shortest message
- D. The window does not count it

3.2 SHORT ANSWER

In a long chat, the oldest turns (where you told the assistant you are vegetarian) were dropped. You ask for a dinner idea. What can go wrong, and name one fix?

3.3 PUT IN ORDER

Put the trimming loop in order: *drop the oldest exchange · count the tokens · stop when under budget · keep the system prompt aside.*

CONCEPT 4 OF 4

Trimming in code, and practical lessons

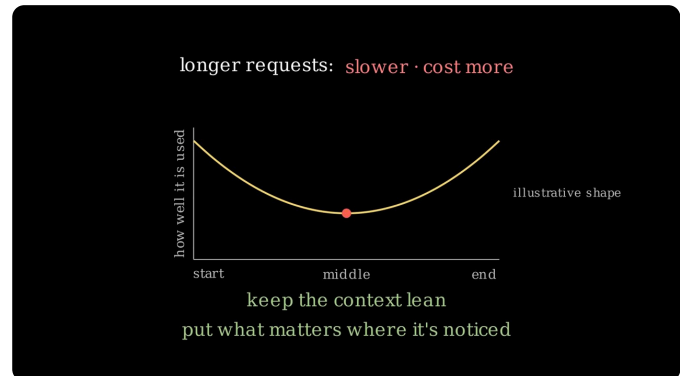
▶ Video 1:44-2:19

In code: count the tokens; while the chat is over budget, drop the oldest exchange, but never the system prompt. In the episode's code a 10-message chat of **133 tokens** is trimmed to fit a budget of **100**: it keeps **82 tokens**, 6 of the 10 messages.

```
1 def fit(messages, budget):
2     system, turns = messages[0], messages[1:]
3     while count(system + turns) > budget:
4         turns = turns[2:] # drop the oldest exchange
5     return system + turns
6
7 kept = fit(chat, budget=100) # 133 tokens -> 82
```

133 tokens → 82 tokens (budget 100)

▶ 1:56 Trimming a 133-token chat to a 100-token budget leaves 82 tokens.



▶ 2:12 Models tend to use the start and end of a long context better than the middle (illustrative shape).

Bigger windows help, but they are not free: long requests are **slower** and **cost more**, and models tend to use information at the **start and end** of a long context better than in the **middle**. So keep the context **lean**, and put what matters most where the model will notice it.

KEY IDEA

Fit the budget deliberately, and spend the window on what matters.

Check 4: Trimming in code, and practical lessons

Answer all 3 questions before looking at the answers. Pass mark: all correct.

4.1 CALCULATION

The trimmed chat keeps 82 of the original 133 tokens. How many tokens were dropped?

4.2 TRUE OR FALSE

“With a 1-million-token window, it no longer matters where in the prompt you put the key instruction.”

True ☐ False ☐

4.3 HANDS-ON CODE

Try it yourself. In `code/p02_context_window/fit_the_window.py`, change `budget = 100` to `budget = 60` and run it. (a) How many messages are kept? (b) Which user question is the first one kept? (c) What happens with `budget = 30`?

```
budget = 60
kept = fit(chat, budget)
print(count(kept), len(kept))
for m in kept:
    print(m["role"], m["content"])
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. The context window · rewatch ▶ 0:08-0:46

1.1 C. The window counts everything in one request, including the tokens the model writes.

1.2 2,768 tokens. $32,768 - 30,000 = 2,768$: the reply shares the window with the prompt.

1.3 9.2. $301,829 / 32,768 \approx 9.21$.

2. Why there is a limit · rewatch ▶ 0:46-1:21

2.1 C. Every token attends to every earlier token, so the work grows with the square of the length: $2^2 = 4$.

2.2 96 MiB. $12,288 \times 8,192 = 100,663,296$ bytes = 96 MiB.

2.3 False. It grows linearly: a fixed amount per token. It is the attention work that grows with the square.

2.4 3,840 MiB (3.75 GiB). 10×384 MiB. This is why long contexts limit how many users a server can handle.

3. When the chat overflows · rewatch ▶ 1:21-1:44

3.1 B. Losing it would change how the model behaves, not only what it remembers.

3.2 It may suggest meat, because that fact is no longer in the context. Fix: summarise old turns (keeping “vegetarian”), or retrieve relevant facts. If it is not in the context, the model cannot see it.

3.3 keep the system prompt aside → count the tokens → drop the oldest exchange → stop when under budget (repeat counting and dropping until it fits). This is the `fit` function of the episode's code.

4. Trimming in code, and practical lessons · rewatch ▶ 1:44-2:19

4.1 51. $133 - 82 = 51$: the two oldest user + assistant exchanges.

4.2 False. Models tend to use the start and end of long contexts better than the middle, and long prompts are slower and cost more.

4.3 (a) 4 messages, 53 tokens: the system prompt, one exchange and the last question. (b) “What about poached eggs?”. (c) Only the system prompt and the last question remain: 2 messages, 28 tokens. The loop drops whole exchanges from the oldest end until the chat fits. It never drops the system prompt or the newest question, so if even those two did not fit, a real app would have to shorten them or reject the request.